



BSc (Hons) in **Information Technology Specialized in AI**
IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P – Performance Measure: The objective evaluation metric that measures how successfully the agent achieves its goals.
E – Environment: The external world or surroundings in which the agent operates.
A – Actuators: The mechanisms (physical or software) the agent uses to perform actions on the environment.
S – Sensors: The hardware or software through which the agent receives information (percepts) from the environment.

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

<code>self.width</code>	-	Defines the width of the game world (grid).
<code>self.height</code>	-	Defines the height of the game world (grid).
<code>self.walls</code>	-	Represents obstacles in the environment.
<code>self.food_positions</code>	-	Represents the locations of food items in the environment.
<code>self.opponents</code>	-	Represents enemy/opponent positions in the environment.
<code>self.agent_pos</code>	-	Represents the agent's current location within the environment.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent

This environment is Multi-Agent because it includes the variable `self.opponents`, which stores multiple opponent agents in the game. These opponents exist alongside the main agent and can interact with or influence the environment and the agent's performance.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of all the percepts (sensory inputs) that an agent has received from the time it was initialized up to the current moment. An intelligent agent makes its decisions based only on this percept sequence and cannot rely on hidden information or future events that it has not yet perceived.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

The `get_percept()` method represents the Sensors (S) component of the PEAS framework.

Because it collects information from the environment and provides the agent with percepts. Sensors are the mechanisms through which an agent receives its percept sequences from the environment.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Fully Observable

Based on the dictionary returned by `get_percept()`, the environment is Fully Observable because the agent receives all the important information needed about the current state of the environment. Since the agent can see the positions of all opponents and receives the relevant state information provided by the environment, there is no indication that important information is hidden from it.

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance Measure (P)

because hitting a wall decreases the agent's score (`self.score -= 5`). The score is the evaluation metric used to measure the agent's performance, so deducting points updates the Performance Measure.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is important because the Performance Measure should evaluate the agent's effect on the external environment, not its internal processing. Penalizing the agent for hitting a wall measures an actual environmental outcome, whereas rewarding or penalizing internal computation could be exploited without accomplishing the task. The lecture states that AI evaluation must measure changes in the external environment state, not the agent's internal processing state.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Hiding `self.toxic_traps` from the agent's sensors changes the environment from Fully Observable to Partially Observable. Although the traps exist in the environment and affect the agent, their locations are not included in the agent's percepts. Therefore, the agent cannot observe all relevant environment states, which matches a Partially Observable environment.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. Implementation Instructions: Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding `smells_toxin` expands the agent's percept sequence by providing additional sensory information about the environment. This helps the agent act rationally because rationality means choosing actions that maximize expected utility based on the information available in the percept sequence. By detecting toxic traps, the agent can avoid dangerous actions and make better decisions using the additional percept data.

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. Implementation Instructions: In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The Vacuum World Exploit was a robot vacuum that received points every time it collected dirt. A rational agent exploited this by cleaning a patch of dirt, dumping it back onto the floor, and cleaning the same spot repeatedly to gain infinite score. This showed that faulty performance metrics can be exploited, so the metric should instead reward keeping the environment clean rather than simply performing the cleaning action.



Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING